

Zurich ServiceNow AI Platform Capabilities

Last updated: January 16, 2026

Some examples and graphics depicted herein are provided for illustration only. No real association or connection to ServiceNow products or services is intended or should be inferred.

This PDF was created from content on docs.servicenow.com. The web site is updated frequently. For the most current ServiceNow product documentation, go to docs.servicenow.com.

Company Headquarters

2225 Lawson Lane
Santa Clara, CA 95054
United States
(408)501-8550

Managing proposed changes

The proposed changes feature allows you to pre-configure changes to configuration items and their associated relationships. These pre-configured changes are prepared to be implemented, but do not actually happen until they are applied at a later time.

When you view a CI, the proposed changes can be displayed so that you can see what is planned.

This feature is useful when you want to make modifications while a change process is in the approval stage, and only implement the changes after the approvals are complete. If the change is never approved, no changes to records have to be reversed. If the change is approved, a quick command applies all the proposed changes.

You can make the following proposed changes to a CI:

- Modify any field on the CI form.
- Add or delete a relationship to that CI.

To modify a relationship, you must delete the current relationship and add a new relationship. You cannot delete a proposed change.

View CI history

You can view the history of changes to a CI in a list, calendar, or timeline format.

View the proposed changes of a CI

You can view the proposed changes so that you can see what is planned for the CI.

Before you begin

Role required: `personalize_form`

About this task

To view any proposed changes, configure the CI form layout to display the **CMDB Scheduled Changes** field. Proposed changes are not displayed in a CI form by default.

Procedure

1. Navigate to **All > Change > Open** and open a change request.
2. In the **Affected CIs** related list, open the **Configuration Item**.
You may also navigate directly to the CI form.
3. Right-click the form header bar.
4. Select **Configure > Form Layout**.
5. Move the **CMDB Scheduled Changes** field to the **Selected** pane.
6. Click **Save**.
The CI form shows the details of any proposed changes in the **Scheduled changes** area.

Add a proposed change to a CI

Proposed changes to a CI can be made while viewing a change request or any task-related record.

Before you begin

Role required: itil

Procedure

1. In the Change Request form, go to the **Affected CIs** related list.
If there are no CIs in the Affected CIs list, click **Edit** to add CIs that are affected by this change request.
2. Right-click the CI that you want to configure for a proposed change, and select **Proposed Change**.
3. Complete the form to make the proposed changes, and click **Save Proposed Change**.

Click **Update** to apply the changes immediately. Click **Delete** to delete the CI.

4. To propose an addition or a removal of a CI relationship:
 - a. Click the plus icon in the **Related Items** section.
 - b. In the Relationships section, add or delete a relationship.
For information about using the relationship editor, see [Create or edit a CI relationship](#).
 - c. Click **Save Propose Change**.
 - d. Confirm saving the proposed change.
Click **Update** or **Delete** to commit the changes immediately.

Note: Use only with CI relationships. Proposing additions or removal of relationships is not valid for user relationships and group relationships.

What to do next

After the proposed changes are saved, the **Apply Proposed Changes** button appears on the Change Request form. This button lets the user commit the proposed changes to the CI. Your business processes determine the appropriate time to commit the changes. The CI retains the existing data until the proposed changes are committed. However, users can see that changes have been proposed.

Apply a proposed change to a CI

When you apply the proposed changes, all the proposed changes for that change request are applied to the configuration item. You can apply proposed changes without verification, or if verification tests of the proposed changes have failed.

Before you begin

Role required: itil

About this task

After you apply the proposed changes, the **Scheduled changes** part of the form displays **No scheduled changes found**. You can configure

proposed change verification rules which you can use to verify proposed changes before applying the changes.

Procedure

1. Navigate to the **Change Request** form.
2. Click the **Apply Proposed Changes** button.
You may have to right-click the form header and select the **Reload Form** option to see the changes.

Create or edit a proposed change verification rule

Ensure that proposed changes meet business requirements and do not introduce invalid data to the CMDB, create a rule that includes a script to verify the proposed changes.

Before you begin

Role required: asset or itil

About this task

When you configure proposed change verification rules for a CI, you have an option to verify that the proposed changes pass the verification test script in the rule. The verification test results are logged as passed or failed, and you can view the results. Running the verification test is not mandatory, and a failed verification test does not prevent you from applying proposed changes.

Procedure

1. Navigate to **All > Configuration > Change Verification > Proposed Change Verification Rules**.
2. Click **New** or select an existing rule to edit.
3. Fill in the fields, as appropriate.

Proposed Change Verification Rules form

Field	Description
Rule name	The name of this rule.
Table name	The table to which the rule applies.
Filter condition	Conditions to apply this rule to specific CIs.
Active	Check box to activate this rule.
Rule script	A verification Java script that needs to return true or false. For example: <pre> validateRule() { var os = current.getValue("os"); var cpu = current.getValue("cpu_count"); //Use current.getValue(fieldName) to get the proposed change value, eg. var os = current.getValue("os"); //Your verification code if (os != "SunOS" cpu < 2) return false; //Return true to pass the verification and false if the verification failed </pre>

Field	Description
	<pre>return true; }</pre>

- Click **Submit** or **Update**.

Result

On the **Change Request** form, you can click **Verify Proposed Changes** to verify proposed changes for the affected CIs.

Verify proposed changes

Before applying proposed changes to affected CIs, use proposed change verification rules to verify that the changes meet business requirements and do not add invalid data to the CMDB.

Before you begin

Create or edit the rules used to verify proposed changes. For details, see [Create or edit a proposed change verification rule](#).

Role required: none

About this task

You can apply proposed changes even if they are unverified or fail a verification test.

Procedure

- Open the **Change Request** form that affects the CI.
- Click **Verify Proposed Changes**.
The proposed changes are verified against any proposed change verification rules in which the CI meets the **Filter condition** criteria.
- Review the message that appears at the top of the form after the verification process is finished.
The message states whether the verification tests passed or failed.

What to do next

To view the details of any verification tests that were performed for the change request in the past two days, click the **Proposed Change Verification Log** related link.

Create or edit a planned change validation script

Create a custom script that checks if a change to a class was valid according to business requirements, and whether the change was planned or not. A planned change validation script is used whenever a CI change is viewed in the CI timeline or change history.

Before you begin

Role required: admin or itil

About this task

The system attempts to validate each CI change as follows:

- If a custom script exists for the CI or one of the CI parents, then the script is executed and the results are used to flag the change as valid or invalid. Parent CIs are examined in the hierarchical order.

-

If a custom script does not exist for the CI or any of its parents, then a predefined validation script is used. The change is determined as a planned change if the change occurred between the **Work start** and **Work end** dates of the change request associated with the changed CI.

However, this check is not always reliable because a user might have manually modified the CI within the work dates, which flags the change as valid even if it is invalid.

The script needs to return a boolean, true or false, which depends on meeting the test criteria in the script. You can define a separate script for each CI class, and you can define multiple planned change validation scripts for a single class. For example, to maintain different versions of the script. Only one script can be active for a CI class at any given time.

These are the parameters that uniquely characterize a change:

- The fields that were changed
- The data source that performed the change
- The time stamp of the change

To correctly determine the validity of a change, examine the parameters and apply business logic to evaluate if the validation tests are met. A planned change validation script can test any of these characteristics and determine when a change meets pre-established criteria. For example, the custom script can check if the mode of the CI is operational or maintenance, or who initiated the change.

Procedure

1. Navigate to **All > Configuration > Change Verification > Planned Change Validation Script**.
2. Click **New** or select a validation script to edit.
3. Complete the form.

Planned change validation script form

Control	Description
Active	Check box to activate this script for validating changes.
Applies to	Class that this script applies to.
Script	Script to run to validate a change. If the script does not return a boolean value, then it is configured to false.

The script has a template which displays the input variables of the script.

Template script input variables

Variable	Type	Description
current	GlideRecord	Current record that is being processed.

Variable	Type	Description
updatedOn	GlideDateTime	Time stamp of the change.
updatedBy	String	Entity responsible for the change.
fieldsChanged	String	Comma-separated list of the names of all fields that were changed.

This sample script checks who initiated the record update. It returns true if admin initiated the record update. Otherwise, the script returns false.

```
isValidChange();

function isValidChange(/*GlideRecord current, GlideDateTime updatedOn, String updatedBy, String changedFields*/)
{
    //Return true if the user that updated the record has an admin role
    return isUserAdmin(updatedBy);
}

function isUserAdmin(userName)
{
    var grUser = new GlideRecord("sys_user");
    grUser.addQuery('name', userName);
    grUser.query();
    if(grUser.next())
    {
        var roles = new GlideRecord ("sys_user_has_role");
        roles.addActiveQuery();
        roles.addQuery('user', grUser.sys_id);
        roles.query();
        while(roles.next())
        {
            if(roles.role.name == 'admin')
                return true;
        }
    }
}
```

```
    }  
  }  
  return false;  
}
```

4. Click **Submit**.